

FleetLogix – Análisis de Arquitectura Cloud en AWS

Avance 4 – Proyecto Integrador

Introducción

FleetLogix continúa su proceso de modernización tecnológica mediante la adopción de servicios en la nube de AWS.

El objetivo de este avance fue diseñar una arquitectura serverless que permita recibir y procesar datos de la flota en tiempo real, almacenar históricos organizados por fecha, calcular métricas operativas (ETA) y generar alertas ante desviaciones de ruta.

Para esto se desarrollaron dos componentes principales:

- ✓ **06_aws_setup_MODO_TEÓRICO**: define la infraestructura de AWS de forma teórica (sin ejecución).
- ✓ **07_lambda_functions_MODO_TEORICO**: contiene las tres funciones Lambda que procesan los eventos de entrega, ubicación y alertas.

Recursos creados y funcionalidades

a) Recursos definidos en 06_aws_setup_MODO_TEÓRICO.py

El archivo 06_aws_setup_MODO_TEÓRICO.py no ejecuta comandos reales, sino que documenta el plan de infraestructura.

Durante su simulación se imprimen los siguientes elementos teóricos:

- **S3 Bucket:** fleetlogix-dev-data
 - *Carpetas:* /raw-data/, /processed-data/, /backups/, /logs/
 - *Funcionalidad:* almacenar datos históricos, procesados y respaldos.
- **Tablas DynamoDB:**
 - deliveries_status (PK: delivery_id)
 - vehicle_tracking (PK: vehicle_id)
 - routes_waypoints (PK: route_id)
 - alerts_history (PK: alert_id)
 - *Funcionalidad:* mantener estado en tiempo real, ubicaciones y alertas.
- **IAM Role:** fleetlogix-dev-lambda-role
 - *Funcionalidad:* permisos mínimos a S3, DynamoDB y SNS para ejecución Lambda.
- **SNS Topic:** fleetlogix-dev-alerts
 - *Funcionalidad:* canal de notificaciones ante desvíos de ruta.
- **RDS PostgreSQL:** fleetlogix-dev-rds-postgres
 - *Funcionalidad:* base de datos analítica con backups automáticos habilitados.

En conjunto, estos recursos definen la capa de almacenamiento y comunicación necesaria para el procesamiento en la nube.

b) Recursos definidos en `lambda_functions.py`

Este script implementa tres funciones Lambda en Python:

1. λ Verificar Entrega:

- Consulta en DynamoDB si una entrega fue completada.
- Devuelve información del estado y hora de entrega.
- Endpoint asociado: `/deliveries/verify`.

• λ Calcular ETA

- Calcula el tiempo estimado de llegada basándose en coordenadas GPS y velocidad.
- Guarda resultados en `vehicle_tracking`.
- Endpoint asociado: `/deliveries/eta`.

2. λ Alerta Desvío:

- Compara ubicación actual con waypoints de ruta esperada.
- Si el desvío supera 5 km, publica una alerta SNS y registra el evento en `alerts_history`.
- Endpoint asociado: `/alerts/deviation`.

Cada Lambda se vincularía al API Gateway mediante métodos POST, dentro de una arquitectura teórica sin necesidad de servidores físicos.

Profundización: función `lambda_verificar_entrega()`

Esta función fue diseñada para verificar si una entrega se completó correctamente en el sistema.

Su flujo lógico es el siguiente:

1. Entrada:

Recibe un evento con `delivery_id` y `tracking_number`.

Si no se proporciona `delivery_id`, devuelve un error 400.

2. Consulta a DynamoDB:

Conecta con la tabla `deliveries_status` y busca el registro por clave primaria.

3. Validación:

Si el registro existe, evalúa si el campo `status` equivale a "delivered".

Si es así, define `is_completed = True`.

4. Salida esperada:

Devuelve una respuesta JSON con el estado actual de la entrega:

```
{
  "delivery_id": 123,
  "is_completed": true,
  "status": "delivered",
  "delivered_datetime": "2025-09-21T15:00:00"
}
```

5. Gestión de errores:

- Si el registro no existe: código 404 ("Entrega no encontrada").
- Si ocurre una excepción de conexión: código 500 ("Error interno del servidor").

Importancia técnica: permite verificar en tiempo real el estado de una entrega desde la app móvil del conductor, reduciendo consultas directas a la base relacional y demostrando cómo una Lambda puede reemplazar una API tradicional en un entorno serverless.

Profundización: función `migrar_datos_postgresql()`

Aunque no se implementa como función real en este avance su rol teórico dentro del archivo `06_aws_setup_MODO_TEÓRICO.py` es crucial en la migración de datos del entorno local hacia la nube.

La función “**migrar_datos_postgresql()**” estaría encargada de:

- ✓ Conectarse al **RDS PostgreSQL** en AWS usando credenciales seguras.
- ✓ Leer y respaldar tablas maestras (conductores, vehículos, rutas).
- ✓ Transferir datos a **DynamoDB** para estado en tiempo real o a **S3** para almacenamiento histórico.
- ✓ Ejecutar validaciones de integridad y registrar logs de sincronización.

Esta integración asegura la coherencia entre la base de datos analítica y los sistemas operativos en la nube, garantizando continuidad con el ETL desarrollado en el Avance 3.

Simulación teórica de `aws_setup_MODO_TEÓRICO.py`

La ejecución local del script (sin conexión a AWS real) generó el siguiente resultado:

== Plan de Infraestructura (Simulación) ==
Región objetivo: us-east-1

Validando nombre S3: fleetlogix-dev-data
OK.

Estructura S3 propuesta:

- **s3://fleetlogix-dev-data/raw-data/**
- **s3://fleetlogix-dev-data/processed-data/**
- **s3://fleetlogix-dev-data/backups/**
- **s3://fleetlogix-dev-data/logs/**

Sugerencia: lifecycle → raw-data/ a Glacier a los 90 días.

Tablas DynamoDB (teóricas):

- **deliveries_status** (PK: **delivery_id**)
- **vehicle_tracking** (PK: **vehicle_id**)
- **routes_waypoints** (PK: **route_id**)
- **alerts_history** (PK: **alert_id**)

IAM Role (Lambdas): fleetlogix-dev-lambda-role

Políticas sugeridas: acceso granular a S3, DynamoDB y SNS.

SNS Topic (alertas): fleetlogix-dev-alerts

RDS PostgreSQL (teórico): fleetlogix-dev-rds-postgres | DB: fleetlogix_dw

Backups automáticos: habilitados (teórico).

Checklist (si fuese despliegue real):

[] Crear bucket S3 + lifecycle
[] Crear tablas DynamoDB
[] IAM role para Lambda
[] SNS Topic y suscripciones
[] RDS PostgreSQL con backups
[] 3 Lambdas y API Gateway (POST)
[] Pruebas con Postman

Esta simulación demuestra la coherencia del diseño sin realizar ninguna creación real de recursos AWS, cumpliendo el objetivo teórico del avance.

Conclusión

La arquitectura propuesta permite que FleetLogix evolucione hacia un entorno **serverless, seguro y escalable**, donde:

- API Gateway actúa como puerta de entrada de las apps móviles.
- Las funciones Lambda procesan datos sin servidores dedicados.
- DynamoDB y S3 almacenan información en tiempo real e histórica.
- SNS envía alertas automáticas ante eventos críticos.
- RDS PostgreSQL mantiene el data warehouse analítico del proyecto.

El diseño teórico garantiza la integración entre todos los componentes de AWS, manteniendo coherencia con los avances previos y sentando las bases para la futura analítica en tiempo real.

Entregables

- ✓ 06_aws_setup_MODO_TEÓRICO.py
- ✓ 07_lambda_functions_MODO_TEÓRICO .py
- ✓ aws_architecture_diagram.png
- ✓ AWS_Análisis_Arquitectura.docx (este documento)